

Day 2 Technical Report

System-Level Reproducibility with Docker & GPU Containers

Hamady Gackou
Master Data Science & AI

February 6, 2026

Abstract

This document reports the engineering work completed on Day 2 of the *ClusterLLM-Local* project. While Day 1 established reproducibility at the **code** and **data** levels using Poetry and DVC, Day 2 addresses a deeper and often overlooked layer: **system-level reproducibility**. We containerize the entire execution environment using Docker and Docker Compose, including the operating system, CUDA runtime, Python interpreter, and dependency installation process. The result is a deterministic, GPU-enabled environment that can be rebuilt and executed on any compatible machine without manual configuration.

Contents

1 From Day 1 to Day 2: Why Containerization Is Necessary

1.1 What Day 1 Did Not Solve

Day 1 successfully locked:

- Python dependencies via `poetry.lock`
- Data versions via DVC

However, the project still depended on:

- the host operating system
- system libraries (libc, SSL, zlib, etc.)
- CUDA runtime versions
- Python interpreter build

These elements are **outside the control of Poetry and DVC**.

1.2 The Remaining Risk

Even with perfect code and data locking:

- a different Linux distribution
- a different CUDA version
- a missing system library

can silently break the project or change results.

Key Observation

Reproducibility is not achieved until the **entire system stack** is controlled.

2 System-Level Reproducibility: Definition

2.1 Formal Definition

System-level reproducibility is the ability for an independent researcher to reconstruct:

- the same operating system
- the same CUDA runtime
- the same Python interpreter
- the same dependency installation procedure

using only declarative configuration files.

2.2 Why It Matters for This Project

- **GPU Dependence:** Results depend on CUDA and cuDNN behavior.
- **Academic Evaluation:** A jury must be able to rerun the system.
- **Longevity:** The project must remain executable months later.

3 Why Docker?

3.1 Rejected Alternatives

- **Manual Setup:** Non-reproducible, undocumented, error-prone.
- **Virtual Machines:** Heavy, non-versioned, hard to audit.

3.2 Docker as Infrastructure-as-Code

Docker allows the entire system to be described as code:

- versioned
- reviewable
- reproducible

Analogy

Poetry locks the ingredients.

Docker locks the kitchen, the oven, and the gas pressure.

4 Dockerfile Design

4.1 Base Image Selection

```
1 FROM nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04
```

Rationale:

- Official NVIDIA-maintained image
- Explicit CUDA and cuDNN versions
- Compatibility with NVIDIA L4 GPU

4.2 Explicit System Dependencies

System libraries are installed explicitly to avoid hidden assumptions:

```
1 RUN apt-get update && apt-get install -y \  
2     build-essential wget curl git ca-certificates \  
3     libssl-dev zlib1g-dev libbz2-dev libreadline-dev \  
4     libsqlite3-dev libffi-dev liblzma-dev
```

4.3 Python 3.13 Built from Source

Ubuntu 22.04 does not ship Python 3.13. Therefore, Python is compiled manually:

```
1 wget Python-3.13.0.tgz  
2 ./configure --enable-optimizations  
3 make altinstall
```

This ensures:

- full control of interpreter version
- independence from OS package repositories

4.4 Ensuring pip Availability

```
1 python3.13 -m ensurepip --upgrade
```

This step is critical: Python compiled from source does not always include pip.

5 Poetry Inside the Container

5.1 Why Poetry Must Live Inside Docker

Installing Poetry on the host is insufficient. The container must be fully autonomous.

```
1 python3.13 -m pip install poetry
```

5.2 Deterministic Dependency Installation

```
1 COPY pyproject.toml poetry.lock ./
2 RUN poetry install --no-root
```

The lock file becomes the **single source of truth** for the Python environment.

6 Docker Compose: Execution and Orchestration

6.1 Why Docker Compose

Docker builds images. Docker Compose defines how they run.

Compose controls:

- GPU access
- volumes
- working directories
- interactive execution

6.2 GPU Exposure

```
1 gpus: all
```

This connects the container runtime to the NVIDIA driver through the container toolkit.

6.3 Volume Mounts

```
1 volumes:
2   - ../:/workspace
3   - ~/.cache/huggingface:/root/.cache/huggingface
4   - ~/.cache/dvc:/root/.cache/dvc
```

These mounts:

- avoid repeated downloads
- separate compute from storage
- improve iteration speed

7 Implementation Log (Day 2)

7.1 Build

```
1 docker compose -f docker/docker-compose.yml build --no-cache
```

Forcing a clean build ensures no hidden dependencies exist.

7.2 Run

```
1 docker compose -f docker/docker-compose.yml run clusterllm
```

Successful execution validates:

- image integrity
- CUDA availability
- correct system configuration

8 Critical Evaluation

8.1 What Day 2 Solves

- OS-level reproducibility
- CUDA reproducibility
- Python interpreter determinism
- elimination of “works on my machine”

8.2 Trade-offs

- long build times (Python compilation)
- larger image size
- increased infrastructure complexity

These costs are intentional and justified by scientific rigor.

9 Conclusion and Transition

Day 2 completes the transition from:

Reproducible code and data

to:

Reproducible system, code, and data

Next Step (Day 3): Experimental pipelines, embeddings, clustering strategies, and evaluation metrics.